

Code Explanation

Code Explanation: AWS Glue PySpark Customer Order Data Processing Job

Overview

This AWS Glue job processes customer and order data from S3, applies data quality rules, registers datasets in the AWS Glue Data Catalog, implements Slowly Changing Dimension (SCD) Type 2 using Apache Hudi, and calculates customer aggregate spending metrics.---

Step-by-Step Breakdown

Step 1: Import Dependencies and Configure Logging

```
import sys, os
from datetime import datetime
from pyspark.context import SparkContext
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql.functions import (col, sum as _sum, count, lit, current_timestamp,
when, trim, lower, upper, to_date)
from awsglue.context import GlueContext
from awsglue.job import Job
from awsglue.utils import getResolvedOptions
import yaml
import logging
```

- Imports PySpark libraries for distributed data processing- Imports AWS Glue-specific libraries for job management- Sets up logging to track job execution---

Step 2: Load Job Parameters

```
def get_job_parameters(): # Loads configuration from YAML file
    config_path = os.path.join(..., 'config', 'glue_params.yaml')
    with open(config_path, 'r') as f:
        params = yaml.safe_load(f)
    return params
```

****Purpose**:** Reads job configuration from `glue\_params.yaml` file containing:- S3 input/output paths- Data quality settings- Hudi configuration- Catalog database/table names****Fallback**:** If config file is missing, `get\_default\_parameters()` provides hardcoded defaults.---

Step 3: Initialize Spark and Glue Contexts

```
def initialize_spark_contexts():
    args = getResolvedOptions(sys.argv, ['JOB_NAME'])
    sc = SparkContext.getOrCreate()
    glue_context = GlueContext(sc)
    spark = glue_context.spark_session
    job = Job(glue_context)
    job.init(job_name, {})
    return sc, glue_context, spark, job
```

****Purpose**:** - Creates SparkContext for distributed processing- Creates GlueContext for AWS Glue-specific operations- Initializes Glue Job for tracking and committing work- Handles both Glue environment and local testing mode---

Step 4: Define Helper Classes for Data I/O

DataReader Class

```
class DataReader:
    def read_csv(self, path, header=True, infer_schema=True,
delimiter=','):
        return self.spark.read.csv(path, header=header,
inferSchema=infer_schema, sep=delimiter)
```

****Purpose**:** Encapsulates CSV reading logic with configurable options.##### **DataWriter Class**

```
class DataWriter: def _write_parquet(self, df, path, mode='overwrite'):
df.write.mode(mode).parquet(path) def _write_hudi(self, df, path, hudi_options):
df.write.format('hudi').options(**hudi_options).mode('append').save(path)
```

****Purpose**:** Provides unified interface for writing data in multiple formats (Parquet, CSV, Hudi).---

****Step 5: Safe Data Reading with Validation****

```
def read_data_safe(spark, path, format_type='csv', **options): if not
validate_s3_path(path): logger.error(f"Invalid S3 path: {path}") return None try:
reader = DataReader(spark) return reader.read_csv(path, ...) except Exception as
e: logger.error(f"Error reading from {path}: {str(e)}") return None
```

****Purpose**:** - Validates S3 path format (must start with `s3://` or `s3a://`)- Handles read errors gracefully- Returns `None` on failure instead of crashing---

****Step 6: Data Quality Processing****

```
def apply_data_quality(df, params): cleaned_df = df # Remove NULL values if
params.get('dq_remove_nulls', True): cleaned_df = cleaned_df.dropna() # Remove
'Null' string values if params.get('dq_remove_null_strings', True): for column in
cleaned_df.columns: cleaned_df = cleaned_df.filter(~(trim(lower(col(column))) ==
'null')) # Remove duplicates if params.get('dq_remove_duplicates', True):
cleaned_df = cleaned_df.dropDuplicates() return cleaned_df
```

****Purpose**:** Applies three data quality rules:1. ****Remove NULL values**:** Drops rows with any NULL columns2. ****Remove 'Null' strings**:** Filters out rows where any column contains the string "null" (case-insensitive)3. ****Remove duplicates**:** Eliminates duplicate rows---

****Step 7: Ingest Customer Data (TR-INGEST-001)****

```
def ingest_customer_data(spark, params): customer_path =
params['inputs_customer_source_path'] customer_df = read_data_safe(spark,
customer_path, format_type='csv', ...) customer_df =
normalize_columns(customer_df) # Convert column names to lowercase return
customer_df
```

****Purpose**:** - Reads customer CSV files from S3- Normalizes column names to lowercase for consistency- Returns customer DataFrame---

****Step 8: Ingest Order Data (TR-INGEST-002)****

```
def ingest_order_data(spark, params): order_path =
params['inputs_order_source_path'] order_df = read_data_safe(spark, order_path,
format_type='csv', ...) order_df = normalize_columns(order_df) return order_df
```

****Purpose**:** Same as customer ingestion but for order data.---

****Step 9: Clean Data (TR-DQ-001)****

```
def clean_data(customer_df, order_df, params): cleaned_customer_df =
apply_data_quality(customer_df, params) cleaned_order_df =
apply_data_quality(order_df, params) return cleaned_customer_df, cleaned_order_df
```

****Purpose**:** Applies data quality rules to both customer and order DataFrames.---

****Step 10: Register Catalog Tables (TR-CATALOG-001)****

```
def register_catalog_tables(customer_df, order_df, params): # Write customer data
to catalog location customer_catalog_path =
params['outputs_customer_catalog_path'] write_data_safe(customer_df,
customer_catalog_path, format_type='parquet', mode='overwrite') # Write order data
to catalog location order_catalog_path = params['outputs_order_catalog_path']
write_data_safe(order_df, order_catalog_path, format_type='parquet',
mode='overwrite') # Register as temporary views
customer_df.createOrReplaceTempView(params['catalog_customer_table_name'])
order_df.createOrReplaceTempView(params['catalog_order_table_name'])
```

****Purpose**:** - Writes cleaned data to S3 in Parquet format at catalog locations- Creates temporary SQL views for catalog integration- Enables querying via AWS Glue Data Catalog---

****Step 11: Implement SCD Type 2 with Hudi (TR-INCR-001)****

```
def implement_scd_type2(order_df, customer_df, params): # Add SCD Type 2 columns
scd_df = order_df.withColumn('IsActive', lit(True)) .withColumn('StartDate',
current_timestamp()) .withColumn('EndDate', lit(None).cast('timestamp'))
.withColumn('OpTs', current_timestamp()) # Configure Hudi options hudi_options = {
'hoodie.table.name': params['hudi_table_name'],
'hoodie.datasource.write.recordkey.field': 'OrderId',
'hoodie.datasource.write.precombine.field': 'OpTs',
'hoodie.datasource.write.partitionpath.field': 'Date',
'hoodie.datasource.write.operation': 'upsert',
'hoodie.datasource.write.table.type': 'COPY_ON_WRITE',
'hoodie.datasource.hive_sync.enable': 'true', ... } # Write to Hudi
write_data_safe(scd_df, hudi_path, format_type='hudi', mode='append',
hudi_options=hudi_options)
```

****Purpose**:** - Adds SCD Type 2 tracking columns: - `IsActive`: Indicates current record (True/False) - `StartDate`: When record became active - `EndDate`: When record was superseded (NULL for current) - `OpTs`: Operation timestamp for conflict resolution- Configures Apache Hudi for incremental data management- Uses `upsert` operation to update existing records or insert new ones- Enables Hive metastore sync for catalog integration---

****Step 12: Calculate Customer Aggregate Spend (TR-AGG-001)****

```
def calculate_customer_aggregate_spend(order_df, customer_df, params): # Calculate
total amount per order order_with_amount = order_df.withColumn('totalamount',
col('priceperunit') * col('qty')) # Aggregate by customer customer_spend =
order_with_amount.groupBy('custid').agg( _sum('totalamount').alias('totalspend'),
count('orderid').alias('ordercount') ) # Join with customer data
customer_aggregate = customer_df.join(customer_spend, customer_df.custid ==
customer_spend.custid, 'inner') .select(customer_df.custid, customer_df.name,
customer_df.emailid, customer_df.region, customer_spend.totalspend,
customer_spend.ordercount) # Write to analytics location
write_data_safe(customer_aggregate, analytics_path, format_type='parquet',
mode='overwrite')
```

****Purpose**:** 1. ****Calculate order totals**:** `totalamount = priceperunit × qty` 2. ****Aggregate by customer**:** Sum total spend and count orders per customer 3. ****Enrich with customer details**:** Join with customer data to add name, email, region 4. ****Write analytics output**:** Save results to S3 for reporting/BI tools---

****Step 13: Main Execution Flow****

```
def main(): # Initialize contexts sc, glue_context, spark, job =
initialize_spark_contexts() try: # Load parameters params = get_job_parameters() #
```

```

Execute pipeline steps
customer_df = ingest_customer_data(spark, params)
order_df = ingest_order_data(spark, params)
cleaned_customer_df, cleaned_order_df = clean_data(customer_df, order_df, params)
register_catalog_tables(cleaned_customer_df, cleaned_order_df, params)
implement_scd_type2(cleaned_order_df, cleaned_customer_df, params)
calculate_customer_aggregate_spend(cleaned_order_df, cleaned_customer_df, params)
job.commit() # Mark job as successful except Exception as e: logger.error(f"Job failed: {str(e)}") raise

```

****Purpose**:** Orchestrates the entire ETL pipeline in sequence with error handling.---

Key Design Patterns

1. ****Configuration-Driven**:** All parameters externalized to YAML file
2. ****Error Handling**:** Safe read/write functions with validation and logging
3. ****Modularity**:** Each requirement implemented as separate function
4. ****Traceability**:** Function names reference technical requirements (TR-XXX-XXX)
5. ****Idempotency**:** Overwrite mode ensures repeatable execution
6. ****Incremental Processing**:** Hudi enables efficient updates without full rewrites---

Data Flow Summary

```

S3 Customer CSV → Ingest → Clean → Catalog (Parquet) ■ → Join → Aggregate → Analytics (Parquet)
S3 Order CSV → Ingest → Clean → Catalog (Parquet) → SCD Type 2 (Hudi) ■

```